GUÍA DE DEFENSA — PROYECTO INTEGRADOR

Sistema de Visión Estereoscópica con ESP32-CAM

Universidad Politécnica Salesiana · Visión Artificial G8

Michael Lata Jordy Romero

Junio 2026

Criterio	Ponderación	Archivo principal
1. Arquitectura de Captura y Hardware	20%	`ESP32_CAM_Stereo.ino, CameraSource.cpp`
2. Calidad del Mapa de Disparidad	25%	`StereoProcessor.cpp`
3. Precisión y Estabilidad de la Distancia	20%	`KalmanDepth.cpp, StereoProcessor.cpp`
4. Efecto de Realidad Aumentada (AR)	15%	`ARRenderer.cpp`
5. Mini-Evaluación Individual (Defensa)	20%	(preguntas clave)
Total	**100%**	

Nota: la ponderación del 85% aplica con cámaras ESP32-CAM-MB/XIAO-SP; este proyecto usa ESP32-CAM AI-Thinker (**20%**).

1 Criterio 1 — Arquitectura de Captura y Hardware (20%)

Lo que pide la rúbrica

Sistema desarrollado en OpenCV/C++, ESP32-CAM con parámetros de firmware ajustados (AEC, AGC, AWB, XCLK), lectura concurrente de ambas cámaras con hilos para reducir latencia.

Firmware ESP32-CAM — `ESP32_CAM_Stereo.ino`

Por qué AEC/AGC/AWB = OFF

SGBM calcula disparidad comparando la intensidad del mismo punto físico en el frame izquierdo y derecho. Si cada cámara ajusta su exposición de forma independiente, ese punto produce valores distintos en L y R → errores sistemáticos en el mapa. Con exposición manual **igual en ambas cámaras** se garantiza paridad lumínica.

Listing 1: Parámetros fijos del firmware (líneas 52–58)

```
constexpr int  CAMERA_XCLK_HZ = 6000000; // 6 MHz - reduce jitter WiFi
constexpr bool CAMERA_AEC     = false;   // AUTO-EXPOSICION OFF -> manual
constexpr bool CAMERA_AGC     = false;   // AUTO-GANANCIA  OFF -> manual
constexpr bool CAMERA_AWB     = false;   // AUTO-BALANCE   OFF -> color fijo
constexpr int  CAMERA_EXPOSURE = 400;    // 0-1200; ajustar por ambiente
constexpr int  CAMERA_GAIN     = 0;      // 0-30; minimo = menos ruido
```

Por qué XCLK = 6 MHz (no 20 MHz)

A mayor frecuencia el sensor demanda más corriente y genera más calor, aumentando el jitter en la transmisión WiFi. A 6 MHz el ESP32 prioriza la integridad del paquete, reduciendo el desincronismo entre frames L y R que degrada la triangulación.

Los parámetros se pueden ajustar **sin reflashear** vía HTTP: `GET /setexposure?v=400` y `GET /setgain?v=0`.

Hilos concurrentes — `CameraSource.cpp`

Listing 2: Hilo productor de frames (HttpCameraSource)

```
// El constructor lanza el hilo worker inmediatamente
HttpCameraSource::HttpCameraSource(std::string url, int timeoutMs)
    : url_(url), timeoutMs_(timeoutMs) {
    worker_ = std::thread(&HttpCameraSource::workerLoop, this);
}
// El hilo descarga JPEG de /capture en bucle continuo
void HttpCameraSource::workerLoop() {
    while (running_.load()) {
        // ... curl descarga JPEG ...
        {
            std::lock_guard<std::mutex> lk(mutex_);
            latestFrame_ = decoded;  // protected write
            hasFrame_    = true;
        }
        framesReceived_.fetch_add(1);
    }
}
// El bucle principal lee sin bloquearse
bool HttpCameraSource::read(cv::Mat& out) {
    std::lock_guard<std::mutex> lk(mutex_);
    if (!hasFrame_) return false;
    out = latestFrame_.clone();
    return true;
}
```

- Cámara izquierda y cámara derecha tienen cada una su propio hilo productor.
- El hilo principal (controlador) consume los frames más recientes sin esperar.
- El `mutex_` protege el acceso al frame compartido (productor/consumidor).
- `std::atomic<bool> running_` permite parar el hilo sin data-race.

2 Criterio 2 — Calidad del Mapa de Disparidad (25%)

Lo que pide la rúbrica

Mapa denso, bordes definidos, bajo ruido. Aplicación justificada de preprocesamiento (CLAHE) y posprocesamiento (WLS) vinculados a SGBM.

Pipeline completo — `StereoProcessor.cpp`

Frame crudo $\xrightarrow{\text{remap}}$ Rectificado $\xrightarrow{\text{cvtColor}}$ Gris $\xrightarrow{\text{CLAHE}}$ Contraste mejorado $\xrightarrow{\text{SGBM}}$ Disparidad Q16.4 $\xrightarrow{\text{WLS}}$ Disparid

Listing 3: Paso 1: Rectificación (corrección de distorsión)

```
// remap aplica los mapas precalculados de calibracion
// para eliminar distorsion de lente y alinear lineas epipolares
cv::remap(rawLeft,  result.leftRect,  calib_->map1x, calib_->map1y, INTER_LINEAR);
cv::remap(rawRight, result.rightRect, calib_->map2x, calib_->map2y, INTER_LINEAR);
```

Listing 4: Paso 3: CLAHE — preprocesamiento (líneas 88–92)

```
// clipLimit=2.0: limita la amplificacion maxima del histograma local
// tileSize 8x8: divide la imagen en teselas para ecualizacion local
clahe_ = createCLAHE(cfg.claheClipLimit, Size(8, 8));
clahe_->apply(grayL, grayL);
clahe_->apply(grayR, grayR);
```

Por qué CLAHE antes de SGBM

SGBM busca correspondencias por similitud de intensidad de bloques. En zonas con iluminación desigual o sombras, el contraste local es bajo y el bloque L no parece el mismo que en R. CLAHE ecualiza el histograma **localmente** sin saturar zonas brillantes, lo que mejora la discriminación en bordes y texturas finas.

Listing 5: Paso 4: SGBM — parámetros clave (líneas 29–35)

```
sgbm_ = StereoSGBM::create(cfg.minDisp, numDisp, blockSz);
// P1 y P2 controlan la suavidad del mapa de disparidad.
// La relacion P2 = 4*P1 es la recomendacion de Hirschmuller.
sgbm_->setP1(8  * 3 * blockSz * blockSz);
sgbm_->setP2(32 * 3 * blockSz * blockSz);
sgbm_->setUniquenessRatio(cfg.uniquenessRatio);
sgbm_->setMode(StereoSGBM::MODE_SGBM_3WAY);
```

Listing 6: Paso 5: WLS — posprocesamiento (líneas 101–108)

```
// El matcher derecho calcula disparidad R->L, requerida por WLS
rightMatcher_->compute(grayR, grayL, disp16R);
Mat filtered;
// WLS rellena huecos y suaviza preservando bordes (guided filter)
wlsFilter_->filter(disp16L, result.leftRect, filtered, disp16R);
filtered.convertTo(result.disparity, CV_32F, 1.0 / 16.0);
```

SGBM vs BM — por qué usamos SGBM

BM (Block Matching) busca solo horizontalmente. SGBM (Semi-Global Block Matching) agrega restricciones de suavidad en 8 direcciones (semiglobal), produciendo mapas más densos y con menos huecos, especialmente en zonas de textura baja.

Listing 7: Paso 6: Relleno de zona inválida (propagación horizontal)

```
// SGBM deja ~numDisp columnas negras a la izquierda sin datos.
// Se propaga el primer valor valido de cada fila hacia la izquierda.
for (int y = 0; y < result.disparity.rows; ++y) {
    float* row = result.disparity.ptr<float>(y);
    float fill = 0.0f;
    for (int x = 0; x < result.disparity.cols; ++x) {
        if (row[x] > 0.5f) { fill = row[x]; break; }
    }
    if (fill <= 0.0f) continue;
    for (int x = 0; x < result.disparity.cols; ++x) {
        if (row[x] > 0.5f) break;
        row[x] = fill;
    }
}
```

Nota de modificación: si se pidiera rellenar **verticalmente** (columna por columna), bastaría hacer el loop externo sobre `x` y el interno sobre `y`, usando `at<float>(y,x)` en vez de `ptr<float>(y)[x]`.

3 Criterio 3 — Precisión y Estabilidad de la Distancia (20%)

Lo que pide la rúbrica

Reproyección correcta de la matriz 3D, coordenada Z en unidades reales (cm/mm), lectura estable con Filtro de Kalman 1D, fluctuación $\leq \pm 3$ cm en entorno estático.

Reproyección 3D

Listing 8: Paso 7: reprojectImageTo3D (línea 131)

```
// Q es la matriz de reproyeccion calculada en la calibracion estereo.
// Convierte el mapa de disparidad en una nube de puntos 3D (X, Y, Z en metros).
reprojectImageTo3D(result.disparity, result.points3D, calib_->Q, true);
```

Listing 9: Medición robusta: mediana de región 10x10 px

```
float StereoProcessor::medianDepthAt(const Mat& points3D, int cx, int cy, int r) {
    // Recolecta todos los Z validos en un cuadrado de radio r
    for (int y = y0; y <= y1; ++y)
        for (int x = x0; x <= x1; ++x) {
            const float z = points3D.at<Vec3f>(y, x)[2];
            // Solo valores finitos en rango plausible (5 cm - 10 m)
            if (std::isfinite(z) && z > 0.05f && z < 10.0f)
                zVals.push_back(z);
        }
    // nth_element: O(n) - equivalente a mediana sin ordenar todo el vector
    std::nth_element(zVals.begin(), zVals.begin() + mid, zVals.end());
    return zVals[mid];
}
```

Filtro de Kalman 1D — `KalmanDepth.cpp`

Listing 10: Filtro de Kalman escalar (las 3 ecuaciones clave)

```
double KalmanDepth::update(double measurement) {
    // PREDICCION: la incertidumbre crece con el tiempo (ruido de proceso q)
    covariance_ += q_;

    // GANANCIA: balancea confianza en prediccion vs. nueva medicion
    const double gain = covariance_ / (covariance_ + r_);

    // ACTUALIZACION: corrige la estimacion con la nueva medicion
    estimate_   += gain * (measurement - estimate_);
    covariance_  = (1.0 - gain) * covariance_;
    return estimate_;
}
```

Variable	Significado	Efecto si sube
`q_`	Ruido de proceso	Kalman confía más en la medición (responde rápido)
`r_`	Ruido de medición	Kalman confía más en la predicción (más suave)
`gain`	Ganancia Kalman	Entre 0 (ignorar medición) y 1 (ignorar predicción)

La tecla **C** reinicia el filtro Kalman central (`kalmanCenter_.reset()`). La tecla **R** graba las mediciones en CSV con columnas `frame, elapsed_ms, z_raw_cm, z_kalman_cm, fps`.

Validación a 30/60/90 cm

El script `python/collect_measurements.py` automatiza la recolección: 50 muestras por distancia, calcula media, desviación estándar y error absoluto, y verifica que `std` ≤ 3.0 cm (criterio de la rúbrica).

4 Criterio 4 — Efecto de Realidad Aumentada (15%)

Lo que pide la rúbrica

Efecto computacional original sobre la transmisión rectificada a color. El efecto debe reaccionar **dinámicamente** a la distancia medida (escala, posición o comportamiento). Diseño exclusivo del grupo.

Descripción del efecto — `ARRenderer.cpp`

El efecto **Energy Ball** tiene tres capas superpuestas, todas moduladas por Z:

Capa 0: Campo de distorsión por profundidad (`applyDepthField`)

Listing 11: Distorsión de onda reactiva a la distancia

```
// Fuerza inversamente proporcional a Z^3: cae muy rapido con la distancia
const float z3       = zFocusMeters * zFocusMeters * zFocusMeters;
const float strength = std::clamp(0.6f / std::max(z3, 1e-4f), 0.0f, 70.0f);
const float speed    = 10.0f / std::max(0.15f, zFocusMeters);

// Desplazamiento senoidal de cada pixel en primer plano
const float ox = strength * std::sin(y * 0.10f + t * speed);
const float oy = strength * std::cos(x * 0.10f + t * speed * 0.75f);
dst[x] = src.at<cv::Vec3b>(sy, sx);  // lee pixel desplazado
```

Comportamiento dinámico: a 20 cm la onda desplaza hasta 70 px a alta velocidad. A 80 cm la perturbación baja a 2 px. A más de 80 cm no hay efecto (`strength < 2.0f`).

Capa 1: Tinte radial (`applyPixelVignette`)

Listing 12: Color del tinte reactivo a la distancia

```
// Color cambia segun la distancia (en BGR):
if (depthCm < 40.0f)       { tB=20; tG=60;  tR=255; }  // ROJO   (peligro: muy cerca)
else if (depthCm < 80.0f)  { tB=20; tG=220; tR=80;  }  // VERDE  (zona optima)
else                        { tB=240;tG=160; tR=20;  }  // CYAN   (lejos)
```

Capa 2: Aura pulsante con destellos orbitales (`drawEnergyAura`)

Listing 13: Pulso y velocidad de destellos reactivos a Z

```
// Pulso a 3 Hz - radio oscila ligeramente
const float pulse  = 0.90f + 0.10f * std::sin(phase * 3.0);
// Mas cerca -> mas destellos y orbita mas rapida
const float proximity = std::clamp(20.0f / depthCm, 0.15f, 1.0f);
const int   sparkCount = 6 + int(8.0f * proximity);  // 6 a 14 destellos
const double spinSpeed = 1.0 + 2.0 * proximity;       // hasta 3x mas rapido
```

Distancia	Color	Comportamiento
< 40 cm (cerca)	Rojo	Onda fuerte, 14 destellos rápidos, tinte intenso
40–80 cm (óptimo)	Verde	Onda moderada, 10 destellos, tinte suave
> 80 cm (lejos)	Cyan	Onda mínima, 6 destellos lentos

5 Criterio 5 — Mini-Evaluación Individual (20%)

Preguntas probables y respuestas clave

1. ¿Qué es la matriz Q y qué hace `reprojectImageTo3D`?

Q es la matriz de reproyección 4×4 calculada por `cv::stereoRectify`. Transforma disparidad d en coordenadas 3D:

$$\begin{bmatrix} X \\ Y \\ Z \\ W \end{bmatrix} = Q \cdot \begin{bmatrix} u \\ v \\ d \\ 1 \end{bmatrix} \quad \Rightarrow \quad Z = \frac{f \cdot B}{d}$$

donde f es la longitud focal y B la línea base (distancia entre cámaras). `reprojectImageTo3D` aplica esta operación a cada píxel del mapa de disparidad, produciendo la nube de puntos 3D almacenada en `result.points3D`.

2. ¿Cómo funcionan los hilos en el sistema?

Cada `HttpCameraSource` lanza un `std::thread` (hilo productor) que descarga JPEG continuamente con libcurl. El hilo principal (consumidor) llama a `read()` que toma el último frame con un `mutex`. Esta arquitectura productora-consumidora permite que las descargas HTTP no bloqueen el procesamiento de SGBM.

3. ¿Cómo se sintonizan los parámetros de SGBM?

- `numDisparities`: rango de búsqueda en píxeles (múltiplo de 16). Mayor = ve más lejos pero más lento.
- `blockSize`: tamaño del bloque de comparación (impar ≥ 3). Mayor = más suave pero pierde detalle.
- `P1, P2`: penalizaciones de suavidad. $P2 = 4 \cdot P1$ es la relación de Hirschmuller. $P2$ alto $\rightarrow$ transiciones suaves entre planos.
- `uniquenessRatio`: rechaza coincidencias si la segunda mejor está a menos del X% de la mejor.
- `speckleWindowSize/Range`: elimina manchas aisladas (ruido de sal y pimienta).

En el sistema se pueden ajustar **en tiempo real** con los trackbars de la ventana de controles sin reiniciar.

4. ¿Qué hace la tecla C?

Reinicia el filtro Kalman central (`kalmanCenter_.reset()`), poniendo `initialized_ = false`. La próxima medición se acepta directamente como nueva estimación inicial, sin arrastrar el historial anterior. Útil cuando el objeto cambia bruscamente de posición.

5. ¿Por qué usar la mediana y no el promedio para medir Z?

La mediana es robusta a valores atípicos (*outliers*). Los píxeles del borde de un objeto tienen disparidades erróneas por la oclusión. El promedio sería contaminado por esos valores; la mediana los ignora siempre que más del 50% de los píxeles en la región sean válidos.

6. ¿Qué diferencia hay entre Z raw y Z Kalman en el HUD?

`Z raw` es la salida directa de `medianDepthAt` (puede variar frame a frame). `Z Kalman` es la estimación filtrada: más estable, reacciona con ligero retraso pero cumple el requisito de fluctuación $\leq \pm 3\,\text{cm}$ de la rúbrica.

7. ¿Por qué el efecto AR actúa solo en el primer plano?

En `applyDepthField` hay un umbral: `const float fgLimit = zFocusMeters + 0.50f`. Solo los píxeles con $Z <$ foco $+0.5\,\text{m}$ reciben la distorsión. Píxeles del fondo (pared, suelo) no se tocan, lo que da sensación de que el efecto envuelve solo al objeto medido.

8. Si te piden cambiar la dirección del relleno (Paso 6) de horizontal a vertical, ¿qué cambias?

El loop externo pasa de `for y` a `for x`, y el interno de `for x` a `for y`. El acceso pasa de `ptr<float>(y)[x]` (eficiente, row-major) a `at<float>(y,x)` (acceso aleatorio, necesario cuando x es el eje externo).

Resumen de archivos por criterio

Archivo	Responsabilidad	Criterio
`ESP32_CAM_Stereo.ino`	Firmware: AEC/AGC/AWB, XCLK, endpoints HTTP	1
`CameraSource.cpp/.hpp`	Hilo productor, mutex, lectura no-bloqueante	1
`StereoCalibration.cpp`	Carga `stereo_calib.yml`, mapas de rectificación, Q	2, 3
`StereoProcessor.cpp`	remap → CLAHE → SGBM → WLS → reproject	2, 3
`KalmanDepth.cpp`	Filtro Kalman 1D escalar	3
`ARRenderer.cpp`	Campo de distorsión, tinte radial, aura pulsante	4
`DisplayManager.cpp`	HUD, trackbars, ventanas	1, 2, 3
`StereoController.cpp`	Bucle principal MVC, CSV logger, teclas	todos
`collect_measurements.py`	Validación 30/60/90 cm, CSV de resultados	3
`find_cameras.py`	Escaneo de red para localizar ESP32-CAMs	1